

C++ 算法笔记

输入输出

```
ios::sync_with_stdio(false);  
cin.tie(0), cout.tie(0);
```

```
cout << fixed << setprecision(1) << a;    // 四舍五入保留小数
```

```
// 无限读入, Ctrl+Z 结束  
while (cin >> n) { ... }
```

数学

```
abs(x);                // 绝对值  
int(ceil(a));          // 向上取整  
swap(a, b);            // 交换
```

```
// 判断质数: 枚举 2 ~ sqrt(n)
```

```
// 向上取整  
int(ceil(a));
```

字符串

```
string s;  
cin.getline(a + 1, n);    // 读入含空格, 下标从 1 开始  
s.size();                 // 长度  
reverse(s.begin(), s.end());  
sort(s.begin(), s.end());  
s.substr(3, 6);           // 下标 3 ~ 6 的子串  
s.substr(1);              // 下标 1 ~ n  
s.find(n);                // 返回下标, 未找到返回 -1
```

数据结构

vector

```
vector<int> vec;  
vec.push_back(5);  
vec.size();  
vec.pop_back();  
vec.clear();  
for (int i : vec) { cout << i << endl; }
```

set (自动排序 + 去重)

```
set<int> s;  
s.insert(x);           // O(log n)  
s.erase(x);           // O(log n)  
s.count(x);            // 判断是否存在, O(log n)  
s.size();              // O(1)  
s.empty();             // O(1)  
*s.begin();            // 最小值  
for (int i : s) { cout << i; }
```

map

```
map<int, int> mp;  
mp[1] = 2;             // 1 -> 2  
mp[1000010000] = 1;    // 不报错  
cout << mp[8];         // 默认 0  
mp.size();             // 映射关系数  
mp.count(9);           // 判断单个映射是否存在  
mp.empty();            // 判断是否为空  
mp.erase(100);         // 删除  
for (auto it : mp) {  
    cout << it.first << it.second;  
}
```

priority_queue

```
priority_queue<int> q;           // 大根堆  
priority_queue<int, vector<int>, greater<int>> q; // 小根堆  
q.push(x);  
q.top();  
q.pop();  
q.size();  
q.empty();
```

struct + 重载运算符

```
struct node
{
    int x, y;
    friend bool operator < (node p, node q)
    {
        if (p.x != q.x) return p.x < q.x;
        return p.y < q.y;
    }
};
```

算法

排序

```
sort(a + 1, a + n + 1);
reverse(a + 1, a + n + 1);
```

二分 / 三分

```
// lower_bound 返回第一个 >= x 的迭代器
*lower_bound(vec.begin(), vec.end(), a[i]) = a[i];

// 三分
l = (r - l) / 3;
```

全排列

```
sort(a + 1, a + n + 1);
do
{
    // ...
} while (next_permutation(a + 1, a + n + 1));
```

二进制枚举

```
for (int i = 1; i < (1 << n); i++)
{
    for (int j = n - 1; j >= 0; j--)
    {
        cout << (i >> j) % 2;
    }
}
```

```
    cout << endl;  
}
```

前缀和

```
// 一维  
p[i] = p[i - 1] + a[i];  
// 区间和: p[r] - p[l - 1]  
  
// 二维  
p[i][j] = p[i-1][j] + p[i][j-1] - p[i-1][j-1] + a[i][j];  
// 子矩阵和: p[x2][y2] - p[x1-1][y2] - p[x2][y1-1] + p[x1-1][y1-1]
```

差分

```
// 区间加: d[l] += x, d[r + 1] -= x  
// 前缀和后得到原数组
```

动态规划

背包

```
// 0-1 背包  
for (int i = 1; i <= n; i++)  
{  
    for (int j = m; j >= v[i]; j--)  
    {  
        f[j] = max(f[j], f[j - v[i]] + c[i]);  
    }  
}  
  
// 可行性背包  
f[j] |= f[j - a[i]];
```

区间 DP

```
for (int len = 2; len <= n; len++)  
{  
    for (int i = 1; i + len - 1 <= n; i++)  
    {  
        int j = i + len - 1;  
        f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + w(i, j));  
    }  
}
```

```
}  
}
```

最长上升子序列

```
// O(n log n)  
for (int i = 1; i <= n; i++)  
{  
    if (vec.empty() || a[i] > vec.back())  
        vec.push_back(a[i]);  
    else  
        *lower_bound(vec.begin(), vec.end(), a[i]) = a[i];  
}
```

错排方案

```
f[n] = (n - 1) * (f[n - 2] + f[n - 1]);
```

图论

DFS

```
bool vis[maxn];  
  
void dfs(int u)  
{  
    if (vis[u]) return;  
    vis[u] = true;  
    for (int v : vec[u])  
    {  
        dfs(v);  
    }  
}
```

BFS

```
int dis[maxn];  
memset(dis, -1, sizeof(dis));  
queue<int> q;  
q.push(1);  
dis[1] = 0;  
  
while (!q.empty())  
{
```

```
int u = q.front(); q.pop();
for (int v : vec[u])
{
    if (dis[v] != -1) continue;
    dis[v] = dis[u] + 1;
    q.push(v);
}
```

近期题目

A - 序列计数 (2026.6.21)

用 ≥ 3 的整数组成 n ，求方案数，模 998244353。

```
f[0] = 1, f[1] = 0, f[2] = 0;
for (int i = 3; i <= n; i++)
{
    f[i] = (f[i - 1] + f[i - 3]) % MOD;
}
```

B - 多边形 (CSP-J 2025)

给定 n 根木棍长度，求能组成多边形的子集数，模 998244353。

```
sort(a + 1, a + n + 1);
dp[0] = 1;
int cur = 0;
for (int i = 1; i <= n; i++)
{
    for (int j = cur; j >= 0; j--)
    {
        dp[j + a[i]] = (dp[j + a[i]] + dp[j]) % MOD;
    }
    for (int j = 0; j <= a[i]; j++)
    {
        ans = (ans + dp[j]) % MOD;
    }
    cur += a[i];
}
```

开发环境

```
// Dev-C++ 设置
// 工具 -> 编译选项 -> 代码生成/优化 -> 语言标准 -> GNU C++ 11
```

代码规范

- `#define int long long, signed main()`
- 全局数组，无函数
- 括号换行（函数、if、for、while）
- `endl` 换行，不用 `\n`
- 输出中文无需额外处理（`-include utf8fix.h`）

Git 工作流

```
git init
git add -A
git commit -m "信息"
# 推送
git remote add origin <url>
git push -u origin main
```

文件夹结构

```
C:\Visual Studio Code\
├── C++\
│   ├── 2025.12.28/      # C++ 代码（按日期）
│   ├── 2026.3.8/       # BFS
│   ├── 2026.3.8-2/     # 综合练习
│   ├── 2026.3.8-2/     # DP
│   ├── 2026.3.22/      # DP2
│   ├── 2026.3.22-2/    # DP3
│   ├── 2026.4.12/      # 背包
│   ├── 2026.4.12-2/    # 背包进阶
│   ├── 2026.4.19/      # DP4
│   ├── 2026.4.26/      # DP5
│   ├── 2026.5.10/      # 小游戏 + DP6
│   ├── 2026.5.16/      # 综合练习
│   ├── 2026.5.24/      # 综合练习
│   ├── 2026.5.31/      # 素数筛、并查集、MST
│   ├── 2026.6.7/       # 约数个数、质因数分解
│   ├── 2026.6.21/      # 序列计数、多边形
│   ├── 笔记.md
│   └── README.md
├── Python/
│   ├── game.py
│   └── 爬虫.py
├── README.md
└── .gitignore
```

杂项

```
for (;;) ;           // 无限循环
unsigned int;        // 正整数
break;               // 跳出
continue;            // 执行下一次
```

```
#define 替换项 替换物
```